

This document is the single source of truth for how to contribute code to LegalVault AI. Read it once end-to-end before you open your first pull request. Every rule here exists because we learned the hard way what happens when it is ignored — merge conflicts, broken tests, and wasted review cycles.

1. Team Roles

Name	Role	GitHub handle	Focus area
Bhaves Khaple	Lead / Reviewer	BhavesKhaple	All modules, merges
Shubham	Backend Dev	—	API, ingestion
Vijay	Frontend Dev	AI-Vijay	SvelteKit UI (7.x tasks)
Tejas	QA / Testing	—	Eval, test suites

Vijay: your open PR (AI-Vijay:adding-my-code) targets main — the wrong base. Retarget it to v1 before review can begin. Steps are in Section 5.

2. Repository Layout

GitHub: github.com/BhavesKhaple/legalvault-ai

Branch	Purpose	Who merges
main	Production-ready milestone releases	Bhaves only
v1	Integration — all PRs target this	Bhaves after review
feature/<task-id>-<slug>	One task, one branch, from your fork	You (PR → v1)

The sequential rule: only ONE task may have Status = In Progress across the whole team at any time. If someone else is In Progress, wait, do a code review, or work on documentation. This is intentional — it prevents merge conflicts and half-finished dependencies reaching v1.

3. One-Time Local Setup

3.1 Fork and clone

- Go to github.com/BhavesKhaple/legalvault-ai → click Fork (top right).
- Clone your fork:

```
git clone https://github.com/<YOUR-USERNAME>/legalvault-ai.git
```

- Add the upstream remote (Bhaves's repo):

```
git remote add upstream https://github.com/BhaveshKhaple/legalvault-ai.git
```

- Verify both remotes exist:

```
git remote -v
```

3.2 Python environment

- Python 3.11+ required. Install it from python.org if missing.
- Create the venv from inside the backend/ folder:

```
cd backend python -m venv venv venv\Scripts\activate # Windows
source venv/bin/activate # macOS / Linux pip install -r requirements.txt
```

- Copy the environment template:

```
cp backend/.env.example backend/.env
# Edit backend/.env – set MODEL_TIER, JWT_SECRET_KEY, etc.
```

3.3 Ollama (local LLM)

- Install Ollama from ollama.com/download.
- Pull the model that matches your hardware tier:

```
ollama pull phi3:mini # Tier 2 – office laptop (default)
ollama pull qwen3:30b # Tier 1 – GPU workstation
```

- Start the server: ollama serve (leave this terminal open).

3.4 Run the backend

```
cd D:\projects\Legalvalult AI\legalvault-ai
backend\venv\Scripts\python -m uvicorn backend.app.main:app --reload --port 8000
```

Folder name 'Legalvalult' has a typo. That is intentional — do not rename it.

4. Daily Contribution Workflow

Step 1 — Pick a task

Open the tracker spreadsheet. Find a task where Status = Ready and Owner is empty. Read the task description, acceptance criteria, and the 'Where in Codebase' column. If anything is unclear, ask in WhatsApp before you start.

Tracker: docs.google.com/spreadsheets/d/1_JkYecHFrZVnllXicZ68Q_sEtrpWVICmhMLb3ZTXtLA

Step 2 — Claim the task

- Set Status = In Progress.
- Set Owner = your name.
- Add today's date to the Started column.

■ **Only one task In Progress at a time across the whole team. Check before you claim.**

Step 3 — Sync v1 from upstream

```
git fetch upstream git checkout v1 git merge upstream/v1
```

Step 4 — Create your feature branch

```
git checkout -b feature/<task-id>-<short-slug>
```

Examples:

```
feature/7.1-upload-ui feature/9.1-backend-dockerfile feature/docling-parser
```

Branch names are lowercase, hyphens only, no spaces. The task ID (e.g. 7.1) must appear first so the tracker can match it.

Step 5 — Write code

- Touch ONLY the files listed in your task's 'Where in Codebase' column.
- Do not clean up adjacent files or refactor things not in your scope.
- Match existing code style — indentation, naming, comment style.
- Type-hint all public functions: `def foo(x: str) -> list[dict]:`
- Write tests in `tests/test_.py` before pushing.

Step 6 — Commit your work

Commit message format (mandatory):

```
<task-id>: <short imperative sentence> Optional body explaining WHY, not what.
```

Examples:

```
7.1: add drag-and-drop upload UI with progress bar  
9.1: backend Dockerfile — multi-stage, non-root user, <1.5GB image
```

- Commit small and often. One logical change per commit.
- Never commit `backend/.env` — it is gitignored and contains secrets.

■ ***If you accidentally commit a secret, tell Bhavesh immediately. Do not try to fix it silently.***

Step 7 — Push to your fork

```
git push origin feature/<task-id>-<slug>
```

Step 8 — Open a pull request

- Go to your fork on GitHub.
- Click 'Compare & pull request'.
- Set base repository: BhaveshKhaple/legalvault-ai, base branch: v1.
- Set head repository: , compare: feature/.

Do NOT target main. Only v1.

5. Pull Request Template

Copy-paste this template into the PR description. Fill every section.

```
## Summary _One paragraph: what this PR does and why._ ## Tracker task - Task ID: (e.g. 7.1) - Tracker row: [link to spreadsheet row] ## Acceptance criteria - [ ] <criterion 1 from tracker> - [ ] <criterion 2 from tracker> ## Files changed _List only the files in the task's assigned folder. If you changed anything outside, explain why._ ## How to test 1. Pull this branch. 2. Run: `pytest tests/<module>` 3. Manually verify: <specific UI action or API call> ## Screenshots (if UI task) _Attach before/after screenshots._ ## Known gaps / deviations _Anything you deliberately left out, and why._
```

5.1 How to retarget an existing PR (Vijay — action required)

Your open PR targets main instead of v1. GitHub does not allow changing the base of a PR to a different repo, so the easiest fix is:

```
# On your local machine, inside your fork: git fetch upstream
git checkout adding-my-code git rebase upstream/v1
git push origin adding-my-code --force-with-lease
```

Then close the old PR and open a new one targeting v1. Paste the PR template above into the new PR description.

6. Code Rules

6.1 What to do

- Python 3.11+. Type hints on all public functions.
- Run `ruff format .` and `ruff check .` before every push.
- Write tests first (or at minimum alongside), never after review.
- Add a `docs/updates/.md` describing what you built, files touched, how to test.
- Keep commits atomic: one logical change per commit.
- Understand every line you push — if you used AI to generate it, be ready to explain it.

6.2 What not to do

- Do not push directly to v1 or main. Ever.
- Do not add a new pip dependency without posting in WhatsApp first.
- Do not change files outside your task's assigned folder without a justification comment.
- Do not commit `.env`, credentials, client PDFs, or any file not tracked by git.
- Do not merge your own PR. Wait for Bhavesh.
- Do not mark a task Done on the tracker until the PR is merged to v1.

7. PR Review Checklist

Bhavesh checks these before merging. Self-review against this list before you request review — it saves at least one round-trip.

Checklist item	Verified by
Only files in the task's assigned folder were touched	Code review
All acceptance criteria from the tracker are met with [x]	Code review
Tests exist and pass locally (pytest)	CI + code review
No secrets, API keys, or client data committed	CI + security scan
Commit messages follow <task-id>: <imperative> format	Git hygiene
docs/updates/<task-id>.md created describing what you built	Documentation
No new dependencies added without WhatsApp group discussion	Dependency audit
AI-generated code fully understood and explained-able in review	Code quality

8. Active RAG Optimization Roadmap

These are the next planned feature branches. Pick from this list when the tracker shows them as Ready. Do not start a roadmap branch before the preceding phase is merged to v1.

Phase	Branch	What	Leads to
1 — Docling parser	feature/docling-parser	Layout-aware PDF/DOCX parsing	Phase 2
2 — Parent-child chunking	feature/parent-child-chunking	Clause child + section parent	Phase 3, 4
3 — Metadata filters	feature/metadata-filters	Date, jurisdiction, version tags	Phase 4
4 — Contextual retrieval	feature/contextual-retrieval	Doc-level BM25 context prefix	Final eval
5 — VLM multimodal	feature/vlm-multimodal	WhisperX, diagram captioning	Other workstation

Phase 5 (VLM multimodal) requires a GPU with ≥ 8 GB VRAM. Do not attempt it on the Legion GTX 1650 laptop. It will be executed on the team's secondary workstation.

9. When You Are Stuck

- Under 30 minutes stuck: try one more thing, then post in WhatsApp with what you tried.
- Over 30 minutes stuck: open a Draft PR and tag Bhavesh with the specific error.
- Never silently abandon a task. Set Status = Blocked on the tracker so it can be picked up.
- If you break something in v1, tell Bhavesh immediately. Do not try to silently fix history.

Questions about task specs, codebase decisions, or whether something is in scope belong in the WhatsApp group — not as GitHub comments after you have already built it. Ask before you build.